

Community Empowerment & Growth Portal (CEGP)

Updated SRS with Micro■Enterprise & Freelance Launcher (MEFL)

Version: 1.0 (Updated with MEFL)

Date: 04 Mar 2026

Prepared for: Academic Use (Under guidance of Asst. Prof. Hitesh Parmar)

Technology: MERN (MongoDB, Express.js, React.js, Node.js)

Table of Contents

1	1. Introduction
2	2. Overall Description
3	3. System Features (Common + Three Modules)
4	4. External Interface Requirements
5	5. System Architecture
6	6. Data Model
7	7. ER Diagram (ASCII)
8	8. Use■Case Diagram (ASCII)
9	9. API Endpoints (Indicative)
10	10. Non■Functional Requirements
11	11. Acceptance Criteria
12	12. Test Plan
13	13. Risks & Mitigations
14	14. Project Plan (6–8 Weeks)
15	15. Deliverables
16	16. Dataset Templates (JSON)

1. Introduction

1.1 Purpose

This SRS defines the updated requirements for the Community Empowerment & Growth Portal (CEGP), a unified MERN platform integrating: (A) Career & Skill Development, (B) Financial Literacy & Personal Budgeting, and (C) Micro■Enterprise & Freelance Launcher (MEFL) to help learners convert skills into income while maintaining sound personal finance.

1.2 Scope (MVP)

Common SSO, unified dashboard, notifications; Career: courses, tests, jobs, mentorship; Finance: transactions, budgets, saving goals, calculators, monthly PDF; MEFL: offerings, leads, orders, invoices (auto■sync to Finance), career recommendations based on offering tags.

1.3 Users & Roles

- Guest – browse public info; register.
- User/Learner – use all modules; save data; get alerts.
- Mentor (optional) – approve mentorship; provide feedback.
- Admin – CMS, users/roles, analytics, imports/exports.

1.4 Constraints & Assumptions

- Mobile■first responsive UI; accessibility mindset.
- English baseline with optional Gujarati/Hindi labels.
- No real payments in MVP; PDFs and data are educational artifacts.

2. Overall Description

2.1 Product Perspective

Single-page React client consuming Express APIs with MongoDB persistence. Shared services for auth, notifications, analytics. Feature flags to enable/disable modules.

2.2 Product Functions (HighLevel)

- Unified login/profile; notifications; dashboard.
- Career: courses, lessons, quizzes, jobs, mentorship.
- Finance: transactions, budgets, saving goals, calculators, monthly PDF/CSV export.
- MEFL: offerings, leads, orders, invoices, auto-sync income to Finance; course recommendations based on offering tags.

2.3 Operating Environment

Client: modern browsers; Server: Node.js (Express); DB: MongoDB Atlas/local; Optional hosting: Vercel/Netlify + Render/Railway.

3. System Features (Common + Three Modules)

3.1 Common & Auth

- Email/password auth; JWT; roles: user, mentor, admin.
- Profile: name, email, phone (opt), language, interests.
- Notifications center; unified dashboard widgets.

3.2 Module A – Career & Skill Development

- Skill library, learning paths; lessons with video embeds.
- Quizzes with scoring & attempts; progress tracking; certificates (optional).
- Jobs board; filters by skill/location/type; saved jobs.
- Mentorship (optional): requests, approvals, session notes.

3.3 Module B – Financial Literacy & Personal Budgeting

- Transactions (income/expense); monthly budgets with alerts.
- Saving goals (target, date, progress); simple EMI/SIP/Inflation calculators.
- Reports: category breakdown, trend charts; CSV export; Monthly PDF.

3.4 Module C – Micro■Enterprise & Freelance Launcher (MEFL)

- Offerings builder (title, scope, duration, price, tags).
- Leads intake; Orders with status flow; client notes/files.
- Quotes & Invoices (PDF); mark paid; receipts; no real gateway in MVP.
- Finance auto■sync: invoice paid → income transaction in Finance.
- Career integration: offering tags → recommended courses.

4. External Interface Requirements

4.1 UI Map

- Public: Home, About, Courses (preview), Jobs (summary), Login/Register.
- User: Dashboard; Career (Courses, Lessons, Quizzes, Jobs); Finance (Tx, Budgets, Goals, Calculators, Reports); MEFL (Offerings, Leads, Orders, Invoices); Notifications; Profile.
- Admin: CMS for courses/jobs; finance tips; offerings templates; users/roles; analytics; import/export; settings.

4.2 Software & Communications Interfaces

- Video embeds (e.g., YouTube unlisted). Cloud storage for images/files (optional).
- REST JSON over HTTPS (prod); CORS scoping to client origin.

5. System Architecture

React SPA $\xrightarrow{\text{i18n/Auth (JWT)}}$ $\xrightarrow{\text{Axios/RTK Query}}$ Express REST API $\rightarrow$ MongoDB
■ PDF/CSV services; cron jobs (optional)

6. Data Model (Conceptual)

6.1 Core Entities

- User, Profile, Notification
- Career: Course, Lesson, Quiz, Progress, Job
- Finance: Transaction, Budget, SavingGoal, FinanceTip
- MEFL: Offering, Lead, Order, Invoice, BusinessLesson

7. ER Diagram (ASCII)

```
+-----+      1      1 +-----+
|  User  |-----| Profile |
+-----+      +-----+
      | 1
      | *
      v
+-----+      1      * +-----+
| Transaction|-----| Budget  |
+-----+      +-----+
      | 1
      | *
      v
+-----+      +-----+
| SavingGoal |      | FinanceTip |
+-----+      +-----+
```

```
User 1---* Progress *---1 Course 1---* Lesson
      |
      *
      Quiz
```

User 1---* Job (saved/apply links)

```
User 1---* Offering 1---* Lead 1---1 Order 1---1 Invoice
  (MEFL)                (status flow)          (paid→Finance.Transaction)
```

8. Use Case Diagram (ASCII)

Actors: [User] [Mentor] [Admin]

[User]

- Register/Login
- View Dashboard (Career, Finance, MEFL widgets)
- Career: Enroll Course, Take Quiz, Save Job
- Finance: Add Transaction, Set Budget, Create SavingGoal, Download Monthly PDF
- MEFL: Create Offering, Capture Lead, Convert Order, Generate Invoice (mark Paid)

[Mentor]

- Review Mentorship Requests
- Add Session Notes

[Admin]

- Manage Users/Roles
- CRUD: Courses/Lessons/Quizzes/Jobs
- CRUD: Finance Tips, Categories
- CRUD: MEFL Offerings Templates, Business Lessons
- Analytics & Exports

9. API Endpoints (Indicative)

Auth & Common

```
POST /api/auth/register
POST /api/auth/login
GET /api/users/me
PATCH /api/users/me
GET /api/notifications
PATCH /api/notifications/:id/read
```

Career

```
GET /api/courses?search=&level=&tag=
GET /api/courses/:id
POST /api/courses (admin)
GET /api/courses/:id/lessons
POST /api/courses/:id/lessons (admin)
GET /api/courses/:id/quizzes
POST /api/courses/:id/quizzes (admin)
POST /api/quizzes/:id/attempts
GET /api/progress/me?courseId=
GET /api/jobs?search=&type=&location=&skill=
POST /api/jobs (admin)
```

Finance

```
GET /api/finance/tx?from=&to=&type=&category=
POST /api/finance/tx
PATCH /api/finance/tx/:id
DELETE /api/finance/tx/:id
GET /api/finance/budgets?month=
POST /api/finance/budgets
PATCH /api/finance/budgets/:id
GET /api/finance/goals
POST /api/finance/goals
PATCH /api/finance/goals/:id
GET /api/finance/reports/monthly.pdf
GET /api/finance/tx/export.csv
```

MEFL

```
GET /api/offerings?mine=true
POST /api/offerings
PATCH /api/offerings/:id
DELETE /api/offerings/:id
POST /api/leads (public)
GET /api/leads?status=
PATCH /api/leads/:id
POST /api/orders
GET /api/orders?status=
PATCH /api/orders/:id
POST /api/invoices
GET /api/invoices?status=
PATCH /api/invoices/:id (mark sent/paid)
GET /api/invoices/:id/pdf
POST /api/pricing/calc
GET /api/biz-lessons?tag=
GET /api/integrations/career/recommendations?offeringId=
(Internal) POST /api/integrations/finance/tx
```

Admin & Analytics

```
GET /api/admin/analytics
```



```
GET /api/admin/users
PATCH /api/admin/users/:id
GET /api/admin/exports/*.csv
```

10. Non-Functional Requirements

- Performance: LCP/TTI < 3s on 4G; pagination; lazy loading.
- Security: JWT, bcrypt, validation, rate limits, CORS scoping.
- Privacy: minimal PII; export/delete my data.
- Accessibility: WCAG 2.1 AA mindset; keyboard navigation; i18n fallback.
- Maintainability: layered architecture; ESLint + Prettier; .env.sample.

11. Acceptance Criteria

- Unified dashboard visible post login with three module widgets.
- Career: complete a lesson & quiz; job listing visible.
- Finance: add tx, set budget & goal; generate monthly PDF.
- MEFL: create offering, capture lead, convert to order, generate invoice and mark paid → income appears in Finance.
- Admin: add a course and a finance tip; add an offerings template; analytics loads.

12. Test Plan (High-Level)

- Unit: auth validators; quiz scoring; budget thresholds; invoice state transitions.
- Integration: course → quiz → progress; tx CRUD → budget alerts → monthly PDF; lead → order → invoice → finance sync.
- Manual: keyboard navigation; mobile viewport; i18n fallback; performance via Lighthouse.

13. Risks & Mitigations

- Content scarcity → seed data & featured lists.
- Manual data errors → edit history & CSV templates.
- Scope creep → lock MVP; payments & chat in Phase 2.

14. Project Plan (6–8 Weeks)

Week 1	SRS finalization, IA, ER/Use Case, wireframes, repo & CI
Week 2	Auth/profile/notifications; base models & seed (Course, Tx, Offering)
Week 3	Career: lessons/quizzes/progress; Jobs board
Week 4	Finance: tx/budgets/goals; calculators; monthly PDF/CSV
Week 5	MEFL: offerings/leads/orders/invoices; finance auto sync; career recos
Week 6	Admin CMS; analytics; i18n; accessibility; QA
Week 7–8	(Optional) Deploy; polish; demo video

15. Deliverables

- Updated SRS (this PDF) + ASCII ER/Use Case diagrams
- Starter Kit ZIP (client + server skeleton, Postman collection, seed JSON)
- Wireframes (optional) + demo video (5–10 min)

16. Dataset Templates (JSON)

Course (example):

```
{
  "title": "Basic Web Development",
  "description": "HTML, CSS, JS for beginners",
  "level": "beginner",
  "tags": [
    "web",
    "frontend"
  ],
  "estimatedHours": 10,
  "status": "published"
}
```

Transaction (example):

```
{
  "date": "2026-03-03",
  "type": "expense",
  "category": "Groceries",
  "amount": 1450,
  "mode": "UPI",
  "notes": "Weekly essentials"
}
```

Offering (example):

```
{
  "title": "Logo Design Starter Pack",
  "description": "3 concepts + 2 revisions + brand sheet",
  "basePrice": 3999,
  "durationDays": 7,
  "tags": [
    "design",
    "branding",
    "logo"
  ],
  "status": "active"
}
```

Note: This SRS includes ASCII diagrams for portability. For production docs, vector diagrams can be added separately.